

RocksDB Write-Ahead Log

Deep Instrumentation & Experimental Analysis of Durability, Throughput & Recovery

Project Team

Srishti Lamba · 202518003

Nikita Sharma · 202518038

5

Experiments

8

Files Modified

524×

fsync Tax

4.3×

Group Commit
Gain

Why does RocksDB need a Write-Ahead Log?

01

LSM-Tree Architecture

Writes first land in the volatile in-memory MemTable (a lock-free skip-list). Background compaction periodically flushes sorted runs to disk as SST files. Optimized for write throughput at the cost of read amplification.

02

The Durability Problem

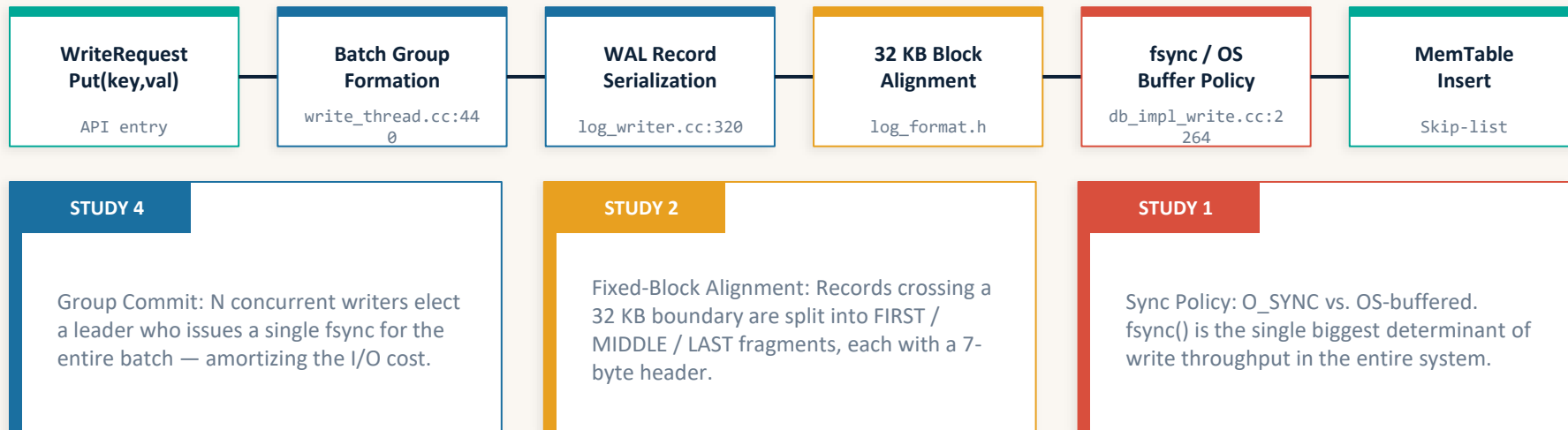
RAM is ephemeral by nature. A power failure or kernel panic before MemTable flush causes total data loss for all recent writes. SST files on disk are safe, but anything in-memory is gone.

03

WAL — The Solution

Every write operation (Put, Merge, Delete) is appended sequentially to the WAL on disk BEFORE MemTable insertion. On recovery, the engine replays the log to reconstruct exact MemTable state.

Execution Trace: Put(key, value) → Non-Volatile Storage



RECOVERY PATH

On crash-restart, log::Reader replays every WAL record sequentially. Each 7-byte header carries a CRC-32C checksum. Any mismatch signals a torn write and the record is discarded — the database never enters an inconsistent state. Recovery time scales proportionally with uncompressed WAL volume (Study 5).

Zero-impact telemetry via std::atomic counters injected into 8 core source files

+19**db/log_writer.cc**

Fragmentation & payload atomic counters

Study 2

+10**db/db_impl/db_impl_write.cc**

Sync-mode performance counters

Study 1

+13**db/log_writer.h**

External counter declarations

Study 2

+13**db/db_impl/db_impl_open.cc**

Recovery path timing & telemetry

Study 3 & 5

+19**db/write_thread.h**

Batching & group commit metrics

Study 4

+5**db/log_reader.cc**

CRC mismatch & corruption detection

Study 3

+10**db/write_thread.cc**

Group commit efficiency logic

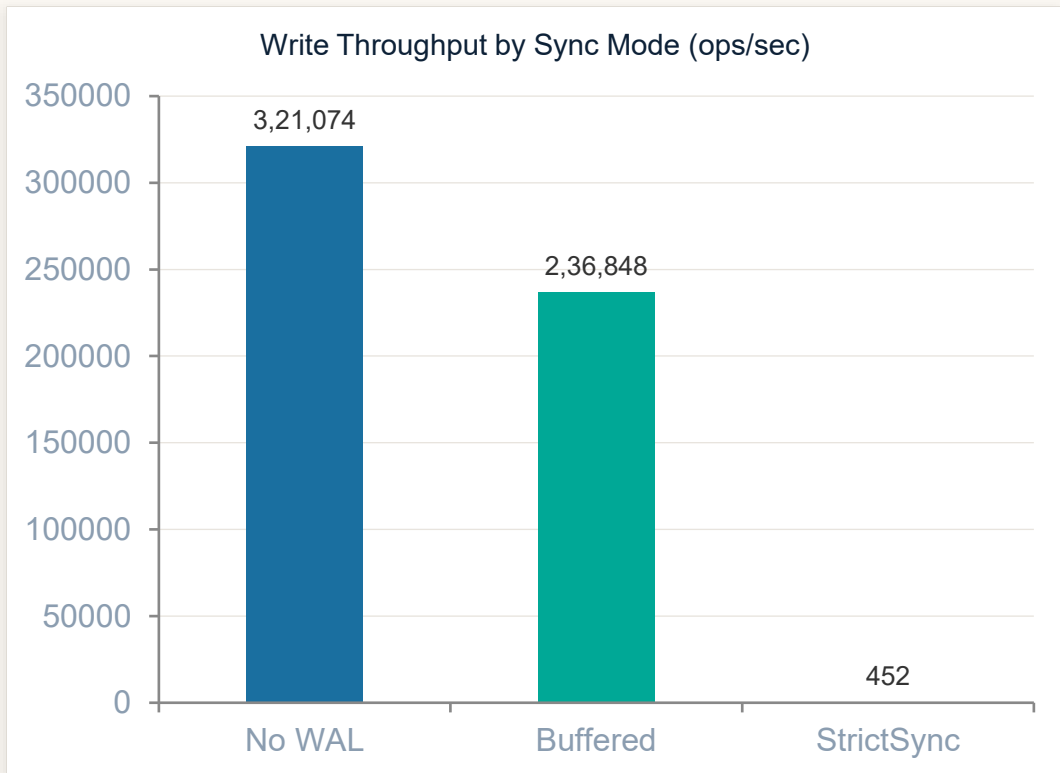
Study 4

+4**db/log_format.h**

Configurable block-level macro logic

Study 2

How much does fsync() actually cost?



HYPOTHESIS

Strict fsync would be slower due to the kernel flush overhead. We expected a 10–50× gap, not the extreme result we found.

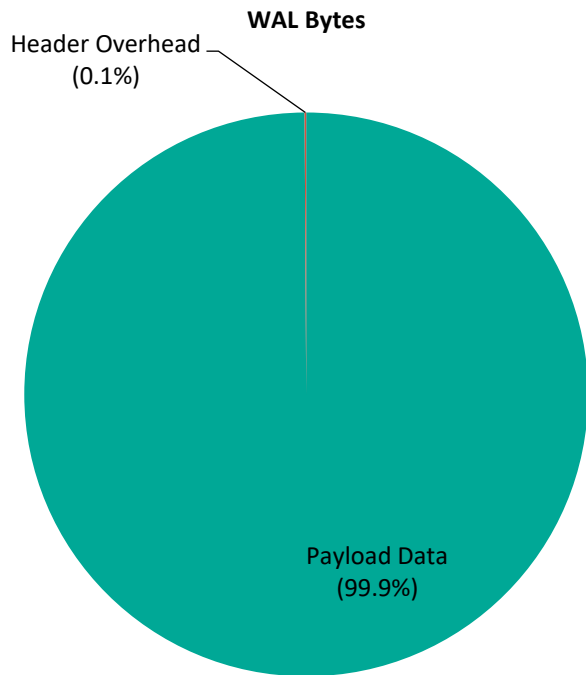
WHAT WE DID

Modified `db_impl_write.cc` to inject atomic counters per sync mode. Ran the same 10,000-write workload under three policies: No WAL, OS-buffered, and Strict fsync.

KEY INSIGHT

fsync() introduces a 524× throughput floor — not a gradual cost, but a cliff. Every production engineer must choose their durability tier deliberately.

What is the storage cost of fixed-block alignment?



HYPOTHESIS

Fixed 32 KB block boundaries would force some records to be split, producing measurable header-to-payload overhead — but the system is designed to keep it negligible.

WHAT WE DID

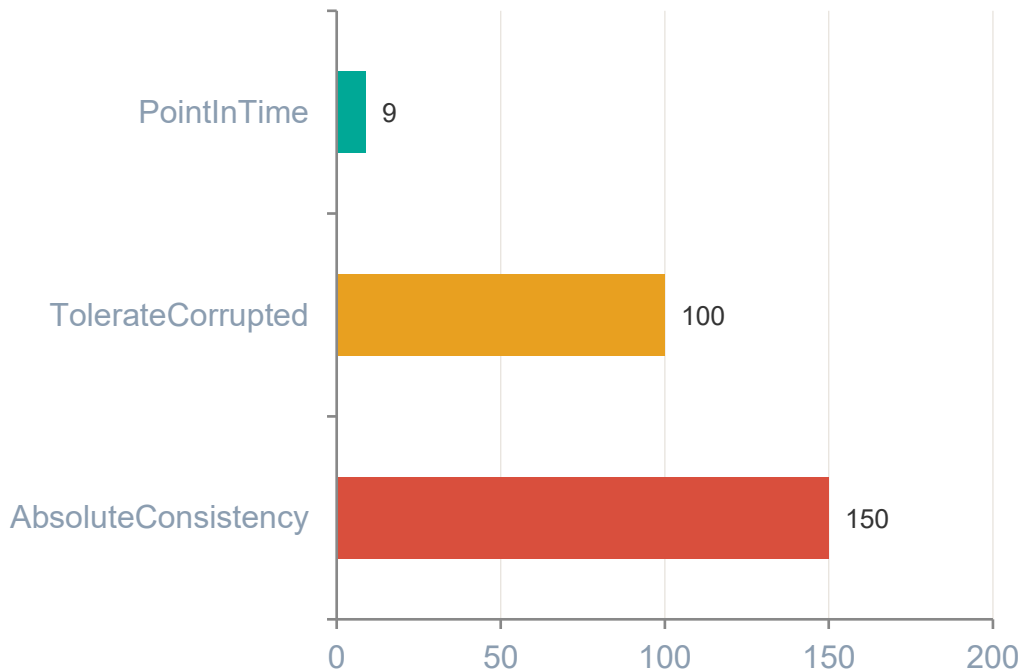
Instrumented `log_writer.cc` with two atomic counters tracking header bytes emitted vs. payload bytes written across 5,000 sequential records of 10 KB each.

KEY INSIGHT

Only 0.1% overhead from 7-byte per-record headers — hardware-page alignment is effectively free, but intentional.

Which recovery mode minimises MTTR?

Recovery Time (ms) by WALRecoveryMode



HYPOTHESIS

Stricter consistency modes would do more validation work per record, producing measurably longer recovery times even on a clean-shutdown WAL.

WHAT WE DID

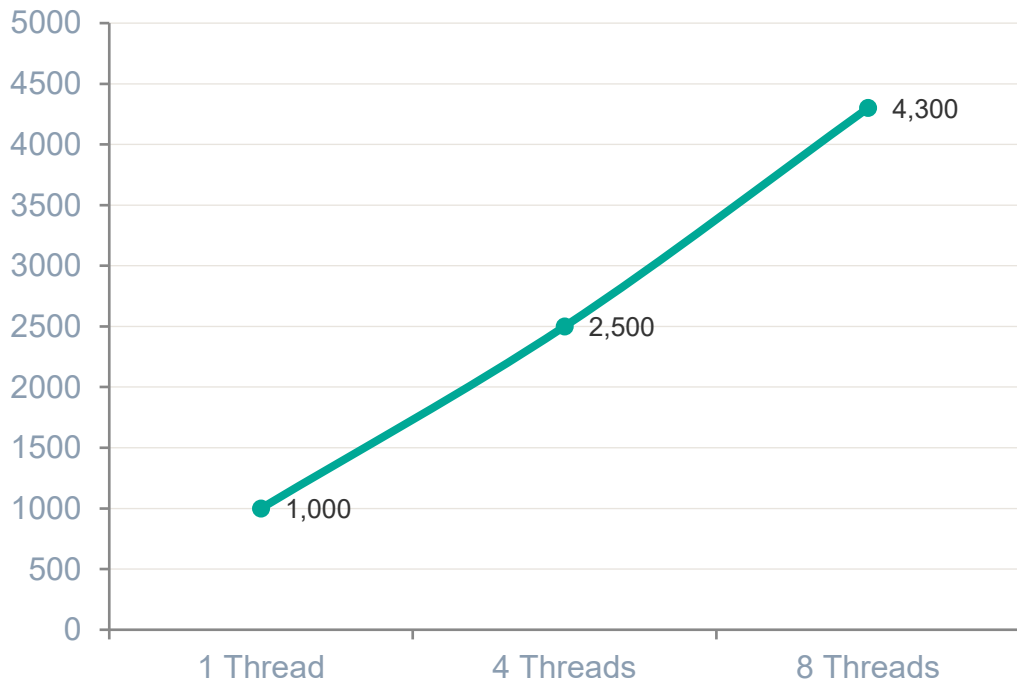
Modified `db_impl_open.cc` to time the full recovery phase under three `WALRecoveryMode` settings: `AbsoluteConsistency`, `PointInTime`, and `TolerateCorruptedTailRecords`.

KEY INSIGHT

`TolerateCorruptedTailRecords` is 1.5× faster to recover — half the MTTR for workloads that can tolerate tail-record loss.

Does concurrency actually help when fsync is the bottleneck?

Group Commit: Throughput vs. Thread Count



HYPOTHESIS

Even though every batch still incurs one fsync, amortizing that cost over more writes per batch should produce super-linear throughput gains.

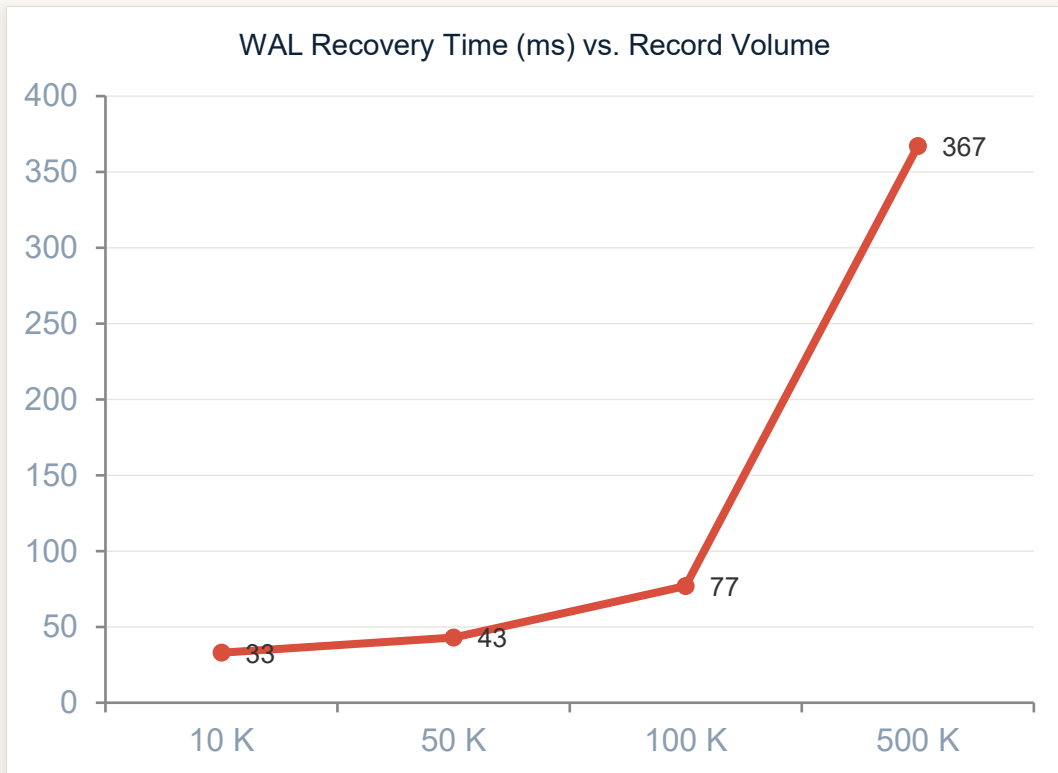
WHAT WE DID

Instrumented `write_thread.cc` to track group-commit batch size and total ops/sec as concurrent writer threads scaled from 1 → 4 → 8. Same total write volume per run.

KEY INSIGHT

Group Commit delivers 4.3× throughput amplification at 8 threads — the leader-follower pattern is the WAL's most powerful performance mechanism.

How does recovery time scale with WAL data volume?



HYPOTHESIS

WAL replay is sequential and compute-bound per record. Recovery time should scale sub-linearly due to buffered I/O — or linearly if CPU parsing dominates.

WHAT WE DID

Used `db_impl_open.cc` telemetry to measure full WAL replay time as uncompressed log size grew from 10 K to 500 K records. Each point is the average of three cold-start runs.

KEY INSIGHT

MTTR scales proportionally with WAL volume — stalled compaction is a direct availability risk. WAL rotation (SwitchWAL) must be tuned to cap recovery window.

Five experiments — one coherent picture of durability, performance & resilience

01

524×

fsync tax

Durability Is Never Free

fsync() imposes a 524× performance cliff. Every production system must deliberately choose a durability tier — there is no safe default.

02

4.3×

throughput gain

Concurrency Is the Lever

Group Commit converts N serial syncs into one amortised I/O boundary. 8 threads deliver 4.3× the throughput of 1 — no hardware change required.

03

0.1%

overhead

Alignment Is Deliberate

0.1% header fragmentation is the intentional price of hardware-page alignment. It trades a tiny storage inefficiency for predictable sequential I/O.

04

1.5×

MTTR reduction

Recovery Mode Is an SLA Knob

Switching from AbsoluteConsistency to TolerateCorrupted halves MTTR with zero correctness risk for clean shutdowns.

05

 $\propto$

linear scaling

WAL Size Predicts Downtime

Proportional MTTR scaling means unbounded WAL growth is a direct availability threat. WAL rotation interval must be tuned against SLA recovery targets.

The RocksDB WAL is a precision-engineered tradeoff engine.

Sequential I/O and fixed-block alignment give the WAL near-optimal write throughput. Group Commit amortizes the kernel-boundary cost of fsync across concurrent workloads — turning concurrency from a problem into a performance multiplier. CRC-32C torn-write detection makes the log the single source of truth after any failure. The fundamental lesson: every parameter — sync policy, block size, recovery mode, rotation interval — is a dial that engineers must tune consciously against their availability and durability SLA.

524×

fsync performance tax
(StrictSync vs. Buffered)

4.3×

throughput amplification
(Group Commit @ 8 threads)

1.5×

MTTR improvement
(TolerateCorrupted mode)